ColdStore-Vision: Guide to Running and Testing

This guide is for anyone who wants to run, test or extend ColdStore-Vision. It covers:

- **What the system does, and how:** the full pipeline, stage by stage, and why each technique was chosen over the alternatives.
- **The codebase:** every Python file, what it does, and which libraries it uses.
- **Hands-on use:** how to try it in 5 minutes with no footage, then on your own recordings.

> Everything below was run on Windows 11 with Python 3.12 on a CPU. Paths are relative to the `coldstore_vision/` folder.

---

1. What the project does

A cold store (godown) holds thousands of onion, potato and garlic sacks on racks. Checking them by hand for rot, settling or theft is slow. ColdStore-Vision needs only two walkthrough videos of the same rack, recorded on different days:

```
Day 1 video (baseline)  ─┐
                         ├──►  per-sack diagnosis  ──►  5 outputs (videos + HTML repo
Day 2 video (inspection) ┘
```

Every sack gets one status:

Status	Colour	Meaning	Suggested action
NORMAL	green	No meaningful change	None
WARNING	yellow	Colour, texture or shape drifted, or the sack was swapped	Re-inspect within 48 h
CRITICAL	red	Rot-coloured area grew sharply, or strong slump	Pull the sack, isolate it

Status	Colour	Meaning	Suggested action
MISSING	cyan	The sack is gone from its spot	Reconcile against the dispatch log
NEW	magenta	A sack appeared that wasn't there on Day 1	Verify against the inward register
UNVERIFIED	grey	The two views were too different, or something blocked the view	Re-scan

UNVERIFIED is deliberate. The system would rather say "I can't tell" than raise a false alarm.

Real result (included in the repo)

`outputs/onion_day1_vs_day2/` comes from two handheld iPhone walks, 21 and 22 Oct 2026, of the onion stack:

- 22 sacks tracked: 4 NORMAL, 4 WARNING (one is a swapped sack), 1 MISSING (unconfirmed) and 13 UNVERIFIED. Most are UNVERIFIED because Day 2 was filmed from a different angle, with a pillar in front.
- Run time was 791 s on a laptop CPU.

---

2. Quick start

2.1 Install

```
py -3.12 -m venv .venv
.venv\Scripts\python -m pip install torch torchvision --index-url https://download.py
.venv\Scripts\python -m pip install -r requirements.txt
```

If you have an NVIDIA GPU, install the CUDA build of torch instead, and set `detector.device: 0` in `configs/pipeline_config.yaml`.

The first run downloads three models, about 1 GB, into `models/hub/`:

- OpenCLIP ViT-B-32, for keyframe retrieval
- LoFTR weights, through kornia
- the YOLOE fallback, used only if `best.pt` is missing

2.2 Try it without any footage (synthetic test, ~2 min)

```
.venv\Scripts\python -m tools.synth.make_pair --out data/synth
.venv\Scripts\python main.py --day1 data/synth/day_01 --day2 data/synth/day_02 --dete
```

`make_pair` builds a fake aisle: 4 bays × 3 tiers. The known changes between Day 1 and Day 2 are:

- one missing sack
- two rotting sacks
- one slumped sack
- one new sack
- a global lighting change

Open `outputs/synth/audit_report.html` and check that every planted change is reported, with no false alarms.

2.3 Run the automated tests

```
.venv\Scripts\python -m pytest -q tests
```

Test file	What it proves
`tests/test_modules.py`	Unit tests for single techniques: • a homography recovers a known warp • histogram matching undoes a lighting change • hue drift is measured correctly on a two-colour background • HSV rot detection and GLCM both react to rot • erosion shrinks masks • Farneback flow sees a downward shift • Hungarian matching pairs sacks correctly • bays and tiers are found from beams
`tests/test_pipeline_synth.py`	End-to-end on the synthetic aisle: • keyframes and registration work • the sack count is right • every planted anomaly is found with no false alarm • depth is used for MISSING • bays and tiers are right

Test file	What it proves
	• the two mosaics align • all outputs are written
`tests/test_polycam.py`	A Polycam "Raw Data" export converts round-trip without losing frames, poses or depth

2.4 Run on real footage

```
.venv\Scripts\python process.py --day1 footage\day1 --day2 footage\day2 --notes "what
```

To reproduce the onion result that ships with the repo, use only the part of each walk after 9.8 s:

```
.venv\Scripts\python process.py --day1 footage\day1\IMG_3866.MOV --day2 footage\day2\
  --day1-start 9.8 --day2-start 9.8 --out outputs\my_onion_run --notes "Onion stack o
```

`process.py` **options:**

Option	Meaning
`--day1 PATH`	Required. A video (`.mp4` / `.mov`), a Polycam Raw Data export (`.zip` or folder), a folder of photos, or a folder holding any of these
`--day2 PATH`	Optional. Leave it out to get a **Day 1 baseline** only (sack count + a recording quality check)
`--dayN-start S` / `--dayN-end S`	Use only part of a video, in seconds. Useful when one walk covers several stacks
`--prefer-lidar`	If a folder has both a video and a Polycam export, use the Polycam one, which adds poses and depth
`--notes "..."`	Ground truth about what you changed. Saved into `run_info.json` so you can score the result
`--skip-video`	Skip the two `.mp4` outputs (faster)
`--out DIR`	Output folder. The default is `outputs/run_<timestamp>/`
`--config FILE`	An alternative YAML config

`main.py` is the older, lower-level entry point. It takes already-prepared day folders and has a `--detector synthetic` switch for the test data.

2.5 How to record your own test footage

To get the most verified sacks:

1. **Same route both days.** Use the same side of the aisle, the same camera height and the same distance from the rack. This is the single biggest factor; different angles are what caused the 13 UNVERIFIED sacks in our run.
2. **Steady, slow walk**, about 0.5 m/s, filming side-on to the rack.
3. **Lock exposure and white balance** (AE/AF lock on iPhone).
4. **1080p or 4K at 30 fps.** All frames are scaled down to 1280 px wide anyway.
5. **Make a known change** between the days and write it in `--notes`. For example, remove one sack, swap one, or put a rotten-looking cloth on one. That is your ground truth.
6. **Optional:** a Polycam **Raw Data** export, which adds LiDAR depth and camera poses. Turn on Settings → Developer Mode in Polycam to see this export. Do *not* use Polycam's "Video" export: it is a rendered fly-through, not your camera footage.

---

3. The outputs

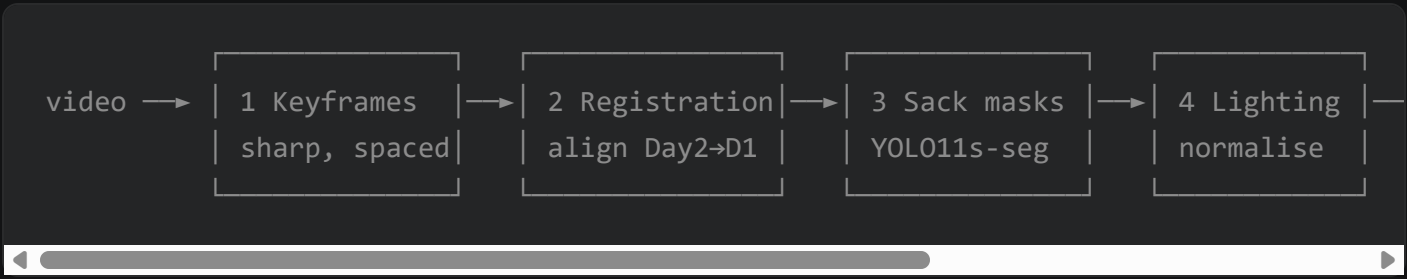
#	File	What it shows
1	`anomaly_video.mp4`	Day 2 replay with every sack outlined in its status colour
2	`audit_report.html` + `manifest.json`	KPI cards and a filterable per-sack table for every sack, plus the same data as JSON. The columns are: • ΔHue • ΔGLCM contrast and homogeneity • rot-area % • ΔZ • flow vy • reason • action
3	`side_by_side.mp4`	Day 1 next to the aligned Day 2, with a banner giving the reason for each flag

#	File	What it shows
4	`rack_digital_twin.html`	Bay × tier grid of the rack. Click a sack for before and after crops and its measurements
5	`panoramic_slider.html`	Day 1 and Day 2 aisle panoramas under a drag-across curtain slider
_	`mosaic_day1.jpg`, `mosaic_day2.jpg`, `run_info.json`	The two panoramas, plus the run's inputs, trims, keyframe counts, KPIs and timing

The HTML files are self-contained, with images embedded as JPEG, so you can email them or open them offline.

---

4. The pipeline, step by step



Orchestration lives in `src/pipeline.py` (`run_inspection`). Every threshold lives in `configs/pipeline_config.yaml`; nothing is hard-coded.

Step 1: Keyframe extraction (`src/ingestion/keyframe_extractor.py`)

What: reduce a video of about 450–510 frames to about 10–20 sharp, evenly spaced keyframes.

How:

1. Read every frame with OpenCV and resize it to 1280 px wide using area interpolation.
2. Score sharpness as the **variance of the Laplacian** of the grey image.
3. Pick candidates in one of two ways:
   - if camera poses exist, every 0.5 m of camera travel
   - otherwise, every 0.5 s
4. If a candidate scores under 100 (blurred), replace it with the sharpest frame within ±5 frames.
5. Attach the LiDAR depth map, if there is one, to each keyframe.

Why these choices:

- **Laplacian variance** is the cheapest reliable blur measure, since blur removes the second-derivative energy. Handheld walks blur during each step, and a blurred frame gives no feature matches and fake texture changes.
- **Travel-distance sampling** gives the same rack coverage whether you walk fast or slow. Time sampling is the fallback when no poses exist.
- **1280 px** is enough to resolve the jute weave and is about 6× faster than 4K. It also puts 1080p and 4K days on the same scale.

Step 2: Registration, or aligning Day 2 onto Day 1 (`src/registration/`)

What: for each Day 1 keyframe, find the Day 2 keyframe that shows the same sacks. Then warp Day 2 so that pixels line up. Also build a panorama of each walk.

How:

- **Pairing** (`robust_matcher.py`, default `matcher: loftr`):
  1. **OpenCLIP** image embeddings shortlist the 4 most similar Day 2 keyframes for each Day 1 keyframe.
  2. **LoFTR**, a transformer-based dense matcher, finds correspondences.
  3. **MAGSAC** fits a homography; a pair needs at least 40 inliers.
- **Classic path** (`homography_warper.py`, `matcher: sift`):
  1. **CLAHE** contrast boost.
  2. **SIFT** features.
  3. **Lowe ratio test** (0.75).
  4. **RANSAC homography** (3 px), followed by a sanity check on the warp.
- **Panorama** (`mosaicking.py`):
  1. Consecutive keyframes are chained with SIFT and an **affine RANSAC** model.
  2. They are blended with OpenCV's **multi-band blender**.
  3. Day 2 is composited onto the *Day 1* canvas, so the two panoramas are pixel-aligned for the slider.

Why these choices:

- **SIFT alone failed** on the real footage. The two days differed in angle, distance, resolution and white balance at the same time. LoFTR matches whole regions rather than isolated corners, so it survives these changes. CLIP retrieval keeps it fast, because LoFTR only runs on 4 candidates rather than all pairs.
- **CLAHE** is used because cold stores are dim. Global equalisation would blow out the lamps, while contrast-limited local equalisation brings out the weave.

- **Homography between days, affine within a walk:** consecutive frames differ only slightly, and affine is more stable there. Across days the viewpoint changes more, which needs the full projective model.
- **RANSAC or MAGSAC** is used because some matches always land on the wrong sack, since every sack looks alike. A robust fit ignores them.

Known limit: a homography is exact only for a flat scene. A sack stack is 3-D, so a different viewpoint leaves **parallax**: sacks near the front shift more than those at the back. Steps 5 and 6 are built to deal with this, using local re-alignment, centroid matching and the UNVERIFIED rules.

Step 3: Sack segmentation (`src/segmentation/sack_detector.py`)

What: an outline (mask), box and confidence for every sack, plus `rack_beam` and `empty_slot`.

How: YOLO11s-seg instance segmentation, trained by us, with weights in `models/weights/best.pt`. If the weights file is missing, it falls back to **YOLOE** open-vocabulary prompts such as "jute sack" or "mesh bag of onions". That works, but less well.

Why:

- Touching sacks of one colour cannot be separated by thresholding, watershed or edges; they merge into one blob. An *instance* segmenter learns where one sack ends and the next begins. On real footage it separated 44 touching sacks in one frame.
- YOLO11s-seg ("small") runs on a CPU at about 1 s per frame and gives polygon masks directly.
- Results: mask mAP50 is 0.80 and precision 0.80 (`reports/training/metrics.json`). The target was 0.85, so a v2 fine-tune on cold-store frames is planned.

See section 6 for how the training data was built.

Step 4: Lighting normalisation (`src/analytics/lighting_normalizer.py`)

What: remove the lighting difference between days so it is not mistaken for colour change, i.e. spoilage.

How:

1. **Histogram specification:** each colour channel of Day 2 is remapped so its cumulative histogram matches Day 1's. The mapping is learnt only from pixels that are not missing or new sacks.
2. **Global hue drift:** the **median** of per-pixel hue differences on the background.

3. Each sack's hue change is reported **relative** to that drift. Hue is averaged as a **circular mean**, because hue is an angle.

Why:

- Day 2 had different lamps and white balance. Without this step, brown jute drifted by about 10° in hue, which looked like rot.
- Masking out missing and new sacks matters because a missing sack would otherwise bend the mapping.
- The median ignores multi-coloured backgrounds, such as blue uprights against orange beams.
- A circular mean is needed because the plain average of 350° and 10° is 180°, not 0°.

Step 5: Matching sacks across days (`src/analytics/tracker.py`, `src/pipeline.py`)

What: decide which Day 2 sack is the same physical sack as each Day 1 sack.

How: the **Hungarian algorithm** (`scipy.optimize.linear_sum_assignment`), an optimal one-to-one assignment, with two possible costs:

- **mask IoU**, when the views are close
- **centroid distance ÷ sack size, plus a size-mismatch penalty** (`match_by_centroid`), when parallax means the masks barely overlap

Before any diagnosis, each matched pair is **re-aligned locally**. The crop around that one sack gets its own SIFT homography (`diagnose_local`).

Why:

- Greedy matching can steal a neighbour's match, whereas Hungarian finds the best overall assignment.
- IoU fails under parallax, while centres stay close, so centroid cost is the default for cross-view footage.
- **Local alignment** fixes what the global homography cannot. At the scale of one sack the surface is nearly flat, so the per-sack warp makes Day 1 and Day 2 crops show the same fabric.

Step 6: Diagnosis (`src/analytics/anomaly_engine.py`, `src/registration/optical_flow.py`)

All measurements are taken **inside the sack mask eroded by 7 px**, never on the whole frame. Edges carry parallax and background, and erosion removes them.

Check	Technique	Triggers
Missing	Masked grey-level difference ($\|\Delta I\| > 40$) over the sack area, with the overlap of any Day 2 sack. With LiDAR, also $\Delta Z > 0.30$ m	MISSING
Rot colour	HSV thresholding: H 10–30, S > 60, V < 110 (dark saturated brown). Measures the fraction of the sack in that range	+3 % gives WARNING, +8 % gives CRITICAL
Colour drift	Relative circular-mean hue change	> 15° gives WARNING
Texture	**GLCM** contrast and homogeneity (32 levels, distance 1, 0/45/90/135°) on pixel pairs inside the mask, after a σ 0.8 Gaussian blur	contrast +35 % or homogeneity −20 % gives WARNING
Slump	**Farneback dense optical flow**: median downward velocity, **divergence** (< 0 means compression), and drop of the sack's top edge	> 4 px gives WARNING, > 10 px gives CRITICAL. Capped at WARNING across different viewpoints
Swapped	Small hue shift together with a large changed area	WARNING ("possibly replaced")

Safety rules, which keep the system from guessing:

- **Occlusion.** A hue jump of more than 90° means something else, such as a pillar, is in front, so the sack is UNVERIFIED rather than MISSING.
- **Pair reliability.** If fewer than 50 % of the sacks in a keyframe pair match, that pair can only produce UNVERIFIED.
- **Truncation.** Sacks cut off by the left or right frame edge, or smaller than 1 % of the frame, never become MISSING.
- **Majority vote.** A sack seen in several keyframe pairs takes its majority status, so one bad view cannot flip a verdict.

Why these techniques:

- **HSV** separates colour (hue) from brightness (value), so a darker lamp doesn't look like rot.

- **GLCM** captures the weave texture. Mould, sprouting and wet patches change texture before they change colour.
- **Optical flow** is used because settling is *motion*: a slumping sack moves down and compresses, which flow and divergence measure directly.

Bays and tiers (`src/analytics/bay_indexer.py`) are inferred without any markers or configuration:

- **Bays:** vertical rack beams split the aisle. With no beams detected, sacks are clustered into columns.
- **Tiers:** horizontal beams split the rows. With none detected, sack rows are clustered by the vertical gaps between them.

---

5. Codebase map

```
coldstore_vision/
├── process.py                one-command entry point (any input format → 5 outputs)
├── main.py                   lower-level entry point on prepared day folders (+ syn
├── configs/
│   ├── pipeline_config.yaml  ALL thresholds and switches for the pipeline
│   ├── dataset.yaml          YOLO dataset definition (classes: sack, rack_beam, emp
│   └── scrape_queries.yaml   web search queries for the training images
├── src/                      the inspection pipeline
├── tools/                    dataset building, remote training, synthetic test data
├── tests/                    pytest suite
├── models/                   train_yolo.py, weights/best.pt (+ .onnx), hub/ (downlo
├── notebooks/train_colab.ipynb
├── data/                     raw/ (prepared day folders), dataset/, scraped/, synth
├── footage/                  the real Day 1 / Day 2 recordings
├── outputs/                  run results
├── reports/                  training metrics, dataset split report
└── video/                    the explainer video project (Blender + HTML + voice-ov
```

5.1 Pipeline (`src/`)

File	What it does	Main libraries
`config.py`	Loads the YAML config and resolves paths	PyYAML
`pipeline.py`	Runs the whole inspection. It pairs keyframes, detects, normalises, matches, runs local diagnosis, takes the majority vote, removes duplicate sightings in mosaic space, assigns bays and tiers, and computes KPIs	NumPy, OpenCV
`ingestion/keyframe_extractor.py`	`FrameSource` reads a video or an image folder. Also handles Laplacian blur scoring, keyframe selection, and loading poses and depth	OpenCV, NumPy
`ingestion/polycam_converter.py`	Converts a Polycam Raw Data export into frames, `camera_poses.csv` and `depth/`	stdlib (json, zipfile), NumPy
`registration/homography_warper.py`	CLAHE, SIFT or ORB matching, Lowe ratio test and RANSAC homography. Also the SIFT keyframe pairing and the warp and valid-mask helpers	OpenCV
`registration/robust_matcher.py`	CLIP retrieval followed by LoFTR plus MAGSAC, for keyframe pairing across days	open_clip_torch, kornia, PyTorch
`registration/mosaicking.py`	Chained affine panorama, multi-band blending (with a feathered fallback) and box projection into mosaic coordinates	OpenCV
`registration/optical_flow.py`	Farneback flow, divergence and per-sack slump measurements	OpenCV, NumPy

File	What it does	Main libraries
`segmentation/sack_detector.py`	Wraps YOLO11s-seg, with the YOLOE fallback. Returns `Instance` objects (class, mask, polygon, box, confidence)	ultralytics
`analytics/lighting_normalizer.py`	Histogram specification, circular hue mean and global hue drift	OpenCV, NumPy
`analytics/tracker.py`	Box and mask IoU, Hungarian IoU matching and centroid matching	SciPy
`analytics/anomaly_engine.py`	Erosion, HSV rot fraction, masked GLCM, the status rules, the reason text and the action	OpenCV, scikit-image, NumPy
`analytics/bay_indexer.py`	Bay and tier assignment from beams, or by 1-D clustering	NumPy
`reporting/output_generator.py`	Calls all the writers below	–
`reporting/video_renderer.py`	`anomaly_video.mp4` and `side_by_side.mp4` (overlays, banners, frame-to-keyframe alignment)	OpenCV (VideoWriter)
`reporting/dashboard_builder.py`	`audit_report.html`, `manifest.json`, `rack_digital_twin.html` and `panoramic_slider.html`, with images embedded as base64 JPEG	OpenCV, stdlib
`reporting/baseline.py`	Day 1-only baseline report: sack count and recording quality check	OpenCV

5.2 Tools (`tools/`)

`tools/dataset/`**: building the training set.** The scripts are listed in the order they run.

File	What it does	Main libraries
`scrape_images.py`	Downloads sack images from Openverse, Wikimedia (CC licences) and Bing	requests, icrawler, Pillow
`prefilter.py`	Removes corrupt and tiny images, removes duplicates by **perceptual hash** (Hamming ≤ 6), and scores relevance with **CLIP** zero-shot	imagehash, open_clip_torch, Pillow
`make_review_batches.py`	Contact sheets and JSON batches for the reviewer agent (`images` , `labels` or a re-review)	Pillow
`merge_reviews.py`	Merges agent verdicts into `accepted.csv` and a class balance report	stdlib
`autolabel.py`	Drafts polygons: **FastSAM** "segment everything", filtered by area and **solidity**, labelled by **CLIP**, merged with **YOLOE**, then simplified to Douglas–Peucker polygons	ultralytics, open_clip_torch, OpenCV
`person_filter.py`	Drops sack polygons that lie on detected people	ultralytics
`render_overlays.py`	Draws numbered masks over images for review	OpenCV
`render_fix_sheets.py`	Overlays with an A–T × 1–12 grid, so the agent can say "add a sack at C4"	OpenCV
`apply_fixes.py`	Applies the agent's delete and split edits, and adds a mask for each new grid point with **SAM 2.1** point prompts	ultralytics (SAM)
`pseudo_label.py`	Adds confident masks from the trained model to the labels (model-in-the-loop)	ultralytics
`cvat_roundtrip.py`	Exports the remaining "fix" images to CVAT for human correction, then imports them back	stdlib (zipfile)
`split_and_package.py`	Leak-free 70/20/10 train, val and test split (near-duplicates by pHash stay in one split), and a zip for upload	imagehash, PyYAML

File	What it does	Main libraries
`validate_dataset.py`	Checks label files: polygon validity, class IDs and image pairs	stdlib
`common.py`	Shared paths and CSV helpers	–

`tools/remote/` : training on a cloud GPU.

File	What it does	Main libraries
`kaggle_train.py`	`all` uploads the dataset and an offline deps dataset, pushes a T4 GPU notebook, waits, then fetches `best.pt` and the metrics	kaggle CLI
`fetch_weights.py`	Copies weights from a Colab download into `models/weights/`	stdlib

`tools/synth/` : test data with known answers.

File	What it does	Main libraries
`make_pair.py`	Builds a synthetic Day 1 and Day 2 aisle with planted anomalies	OpenCV, NumPy
`make_real_pair.py`	Makes a test pair from a *real* frame: removes or swaps a sack and fills the gap realistically	OpenCV
`synthetic_detector.py`	A colour-key "detector" for synthetic data. It uses morphological opening and closing, then connected components, so the pipeline can be tested without YOLO	OpenCV

Other folders:

- `models/train_yolo.py` is the training script that runs on Kaggle or Colab. It trains yolo11s-seg for 150 epochs at imgsz 640, with AutoBatch.
- `tests/conftest.py` holds the shared fixtures: config, the synthetic pair, and one cached inspection run.

5.3 Explainer video (`video/`, optional)

File	What it does	Libraries
`blender/build_scene.py`	Builds the procedural cold-store and drone concept scene	Blender 4.1 `bpy`
`web/*.html`, `style.css`	Prototype and results scenes, animated in HTML/CSS/SVG	–
`render_html.py`	Renders the HTML scenes frame-exact to MP4	Playwright, ffmpeg
`assemble.py`	Cuts the footage clips and joins everything together	ffmpeg
`voiceover.py`	Neural TTS narration, timed to each scene and mixed at −16 LUFS	edge-tts, ffmpeg

These need extra packages that are not in `requirements.txt`: `pip install playwright edge-tts`, then `playwright install chromium`. They also need ffmpeg and Blender. The full workflow is in `.claude/skills/coldstore-video/SKILL.md`.

---

6. How the detector was trained (and how to retrain it)

There was no cold-store footage at the start, so the dataset was built from the web. AI review and segment-anything tools did most of the labelling work:

```
scrape → prefilter (pHash, CLIP) → agent image review → autolabel (FastSAM+CLIP+YOLOE
      → agent label review → agent "fix" points → SAM 2.1 masks → re-review → (CVAT f
      → split 70/20/10 → train yolo11s-seg on Kaggle T4 → best.pt
```

The full command list is in `README.md` under "Step 0". To retrain on Kaggle:

1. Put your API token in `%USERPROFILE%\.kaggle\kaggle.json`. Your phone number must be verified for GPU access.
2. Run:

```
.venv\Scripts\python -m tools.remote.kaggle_train all
```

Why this approach:

- **Agent review:** labelling 146 dense images by hand would have taken 8–15 h. Reviewer agents point at the mistakes, and SAM draws the masks.
- **Kaggle:** it gives a free T4 GPU and can be driven entirely from the command line.
- **yolo11s-seg:** the best trade-off between speed and accuracy for CPU inference in the godown.

Next step: fine-tune a v2 model on auto-labelled Day 1 frames from the godown itself, to reach the 0.85 mAP gate.

---

7. Tuning and troubleshooting

Symptom	Likely cause	What to change
Most sacks UNVERIFIED	Day 1 and Day 2 filmed from different angles or distances	Re-record on the same route (section 2.5). Raising `tracking.centroid_max_cost` accepts looser matches, at the cost of more risk
"best.pt not found – using YOLOE fallback"	No trained weights	Put `best.pt` in `models/weights/`
Few or no keyframe pairs	Too little overlap, or a very dark scene	Check that both walks cover the same stack. Use `--dayN-start/--dayN-end` to trim to it, or lower `registration.loftr_min_inliers`
Too many colour WARNINGs	Lighting changed a lot, or white balance wasn't locked	Keep `lighting.histogram_match: true`, lock WB when recording, or raise `anomaly.hue_warning_deg`
False slump flags	Viewpoint parallax	Expected across views; it is already capped at WARNING. Same-route footage removes it
Output videos don't play in the browser	The `mp4v` codec	Install openh264 and put `avc1` first in `outputs.codecs`

Symptom	Likely cause	What to change
Very slow	4K or long videos	Trim with `--dayN-start/--dayN-end`, use `--skip-video`, or switch to a GPU (`detector.device: 0`)
LoFTR or CLIP download fails	Offline machine	Set `registration.matcher: sift` to use the classic path, which needs no download

8. Library summary

Library	Used for
OpenCV (`opencv-contrib-python`)	Video I/O, colour conversion, CLAHE, SIFT and ORB, homography and affine fitting with RANSAC or MAGSAC, warping, Farneback flow, morphology, contours, multi-band blending, drawing, JPEG and MP4 writing
NumPy	All array maths: masks, histograms, CDF mapping, flow statistics
SciPy	Hungarian algorithm (`linear_sum_assignment`)
scikit-image	GLCM (`graycomatrix`)
ultralytics	YOLO11s-seg training and inference, YOLOE, FastSAM, SAM 2.1
PyTorch / torchvision	Backend for YOLO, CLIP and LoFTR
kornia	LoFTR dense matcher
open_clip_torch	CLIP image embeddings (retrieval) and zero-shot image filtering
imagehash, Pillow	Perceptual-hash deduplication and image loading
albumentations	Extra training augmentations (grey-scale, blur)
requests, icrawler	Image scraping
kaggle	Remote GPU training
PyYAML, tqdm	Config and progress bars
pytest	Test suite

9. Checklist for testers

- ☐ Install (2.1) and run `pytest -q tests` Install (2.1) and run `pytest -q tests`: every test passes.
- ☐ Synthetic demo (2.2): every planted anomaly is in `audit_report.html` Synthetic demo (2.2): every planted anomaly is in `audit_report.html`.
- ☐ Reproduce the onion run (2.4): 22 sacks, with a status mix close to 4 / 4 / 1 / 13.
- ☐ Record your own pair (2.5) with one known change, run `process.py --notes` Record your own pair (2.5) with one known change, run `process.py --notes`, and check that the change is flagged.
- ☐ Open all 5 outputs and look for false alarms. Report the sack ID and the reason text.